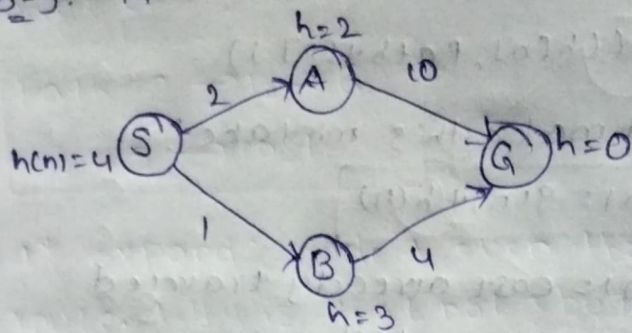


lab-5: A* Search



A* uses,

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start to currNode

$h(n)$ = estimated cost from currNode to Goal

$f(n)$ = estimated total cost of going through that node.

We have,

$$\text{true-cost} = \{ 'S': 5, 'A': 10, 'B': 4, 'G': 0 \}$$

$$\left. \begin{array}{l} S \rightarrow G = 5 \\ A \rightarrow G = 10 \\ B \rightarrow G = 4 \\ G \rightarrow G = 0 \end{array} \right\} \begin{array}{l} \text{Actual} \\ \text{cheapest remaining} \\ \text{cost.} \end{array}$$

for node in h:

ok = "OK" if $h[\text{node}] \leq \text{true-cost}[\text{node}]$ else "VIOLATION"
 Print (f" h[{node}] = {h[node]} true = {true-cost[node]} {ok}")

it is checking,

$$h(n) \leq \text{actual remaining cost}$$

For our heuristic,

$$h(S) = 4 \leq 5 \rightarrow \text{OK}$$

$$h(A) = 2 \leq 10 \rightarrow \text{OK}$$

$$h(B) = 3 \leq 4 \rightarrow \text{OK}$$

$$h(G) = 0 \leq 0 \rightarrow \text{OK.}$$

} → called
admissible heuristic

- By this, heuristic never overestimates the actual remaining cost

queue [(h[start], 0, [start])]

↓

stores $2 = (f, g, \text{path})$

Initially, we are at S

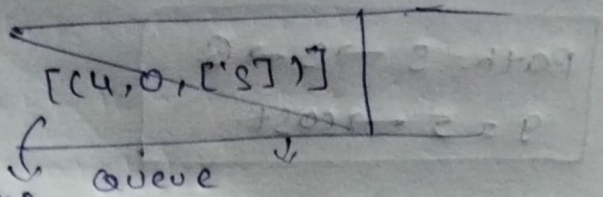
For S:

$$g(S) = 0$$

$$h(S) = 4$$

$$f(S) = g + h$$

$$= 0 + 4 = 4$$



— Again at every step sort the Queue to take smallest cost one & continue this until Q is empty.

$$S \neq \text{Goal}(G)$$

Expand S.

cost, A

$$S \rightarrow A = 2$$

$$g(A) = 2$$

$$h(A) = 2$$

$$f(A) = 2 + 2$$

$$= 4$$

For B,

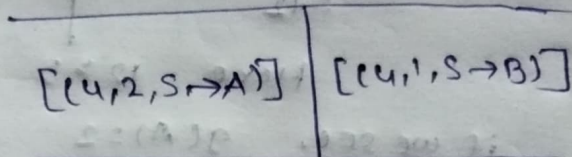
$$S \rightarrow B = 1$$

$$g(B) = 1$$

$$h(B) = 3$$

$$f(B) = 1 + 3$$

$$= 4$$



$$\text{node} = B \neq \text{Goal}(G)$$

Pop()

Queue smallest

Here both

have 4 but in tuple order $\min(1, 2)$ is min so, we take B.

Expand B.

$$S \rightarrow B = 1$$

B has neighbor G with cost 4

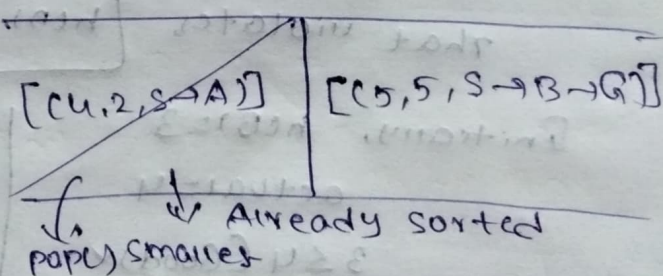
$$g(G) = 1 + 4$$

$$= 5$$

$$h(G) = 0$$

$$f(G) = 0 + 5 = 5$$

$$\text{node} = A \neq \text{Goal}(G)$$



Expand A.

$$A \rightarrow G = 10$$

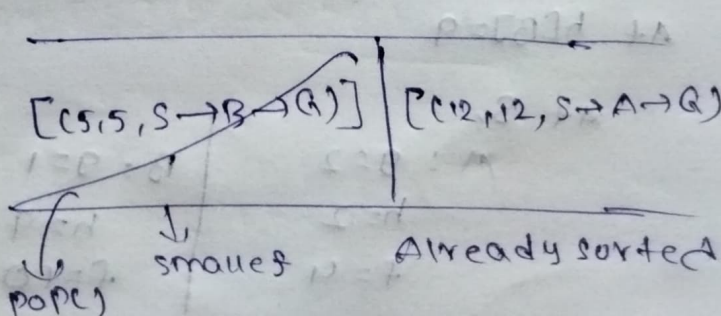
$$g(G) = 2 + 10$$

$$= 12$$

$$h(G) = 0$$

$$f(G) = 12 + 0$$

$$= 12$$



node = G = goal(G) ✓

Path = S → B → G

g = 5 → cost

Q: A* expands node A even though A is on the bad route. Why is that not a bug?

S → A → G
Cost = 2 + 10
= 12

S → B → G = 5

Here, $12 > 5$ but,

if we see, $g(A) = 2$
 $h(A) = 2$
 $f(A) = 4$

$g(G) = 5$
 $h(G) = 0$
 $f(G) = 5$ } through B.

Here A* chooses the smallest f.

Expanding a node ≠ choosing its path as final answer

If we change, $h(B) = 9$

Actual cost from B → G = 4

But $h(B) = 9$

So, $g > 4$ → it is overestimating.

That violates

$$h(n) \leq h^*(n)$$

Initially, $h(B) = 3$

actual = 4

$3 \leq 4$ (Good) ✓

After changing,

$h(B) = 9$

actual = 4

$9 > 4$ (Bad) ✗

So, A* thinks B is worse

At $h(B) = 9$

At S,

A: $g = 2$

$h = 2$

$f = 4$

B: $g = 1$

$h = 9$

$f = 10$

So, the Queue becomes

A* chooses A.

After A

$S \rightarrow A \rightarrow G$

$cost = 12$

The goal has

$f = 12$

But B has

$f = 10$

So, eventually it
expands B.

Finally, $S \rightarrow B \rightarrow G = 10$

